Probability of Default MoC Type C Quantification

The R Package pd.moc.c

Andrija Djurovic

Contents

1 Margin of Conservatism in Probability of Default Modeling . . . 1

2 Practical Challenges in Quantifying MoC Type C . . . 2

3 The R Package pd.moc.c . . . 2

4 Case Study . . . 3
- 4.1 Simulation Setup . . . 3
- 4.2 Statistical Method 1 . . . 3
- 4.3 Statistical Method 2 . . . 4
- 4.4 Statistical Method 3 . . . 5
- 4.5 Statistical Method 4 . . . 5
- 4.6 Statistical Method 5 . . . 6
- 4.7 Statistical Method 6 . . . 6
- 4.8 Statistical Method 7 . . . 7
- 4.9 Statistical Method 8 . . . 7
- 4.10 Statistical Method 9 . . . 8

5 Conclusions . . . 9

1 Margin of Conservatism in Probability of Default Modeling

After completing Probability of Default (PD) model development, practitioners typically address overall model uncertainty by defining a so-called Margin of Conservatism (MoC).

The European Banking Authority (EBA) defines three categories corresponding to distinct sources of uncertainty in the estimated parameters:

1. Type A: uncertainty related to data and methodological deficiencies;
2. Type B: uncertainty related to changes in risk appetite and internal processes;
3. Type C: general estimation error, i.e., statistical uncertainty arising from parameter estimation.

One of the most commonly applied MoC Type C relates to uncertainty in estimating the long-run average (LRA) default rate (central tendency). Uncertainty related to the LRA is the main focus of this R package.

According to the EBA PD & LGD Guidelines, for the purpose of determining PD estimates in the calibration process, institutions may consider either the long-run average default rate at the grade (pool) level or at the calibration segment level. Accordingly, a MoC Type C is expected to be quantified.

In line with the EBA's requirements, the ECB, in its Guide to Internal Models (2025), paragraph 327, section *Model-Related MoC*, clarifies its understanding of the quantification of MoC Type C, emphasizing

that specific sources of uncertainty should reflect the dispersion of statistical estimators. Furthermore, the ECB distinguishes between models that output rating grades or pools and models with continuous outputs. As this `R` package focuses on statistical approaches applicable to the most common PD modeling framework, where the final model output is a rating scale, the remainder is dedicated to the ECB's interpretation in this context.

For models with rating grade outputs, the ECB emphasizes two main sources of uncertainty:

1. the average of one-year default rates over time;
2. the number of exposures per rating grade and per year, which affects the uncertainty of the one-year default rate.

Additionally, with regard to the average of one-year default rates, the ECB notes that dependency between default rates over time should be considered in the MoC Type C quantification process.

2 Practical Challenges in Quantifying MoC Type C

The ECB has set clear expectations for MoC Type C quantification, stating that the fewer the observations per grade and the shorter the time series, the higher the Type C assigned to that grade should be. Additionally, the ECB expects that the correlation between yearly default rates is properly addressed in this process.

In practice, institutions often use simplified frameworks for MoC Type C quantification, typically neglecting either the correlation structure of default rates over time, the variability within specific reference dates, or both. In some approaches, the variability of default rates over time is also not considered.
Considering some or all sources of uncertainty, combined with different statistical approaches, can result in significantly different MoC Type C levels being added to the PD best estimate. Therefore, some of the main challenges can be summarized as follows:

- Given the significant differences in MoC Type C levels depending on the statistical approach and underlying assumptions, is there a single "best" method to properly address the quantification process?
- Can practitioners set a "reasonable" level of MoC Type C for a modeled portfolio type?
- How can practitioners balance regulatory requirements with a reasonable level of MoC Type C?
- Is it always reasonable to add MoC Type C to the best estimate?
- Can MoC Type C quantification rely entirely on a statistical approach?

To address some of the above challenges, practitioners should have a clear understanding of the different statistical methods used for quantifying MoC Type C, their underlying assumptions, and the contribution of different sources of uncertainty to the overall MoC levels.

Thus, the main purpose of this package is to provide an easy way to run different statistical approaches for quantifying MoC Type C, allowing practitioners to gain better insights into this modeling task.

3 The `R` Package `pd.moc.c`

The `R` package `pd.moc.c` provides a framework for quantification of PD MoC Type C related to the uncertainty of the long-run average (LRA) default rate.

The development version is currently available on this GitHub page.

To install the `pd.moc.c` package, run:

```
devtools::install_github(repo = "andrija-djurovic/adsfcr",
                         subdir = "risk_quantification/pd_moc/r",
                         upgrade = "never")
```

After installing the package, load it with:

```r
library(pd.moc.c)
```

The package currently includes nine functions (methods) for quantifying MoC Type C. Each function name begins with `odr`, `lra`, or a combination of both, indicating the primary source(s) of uncertainty it addresses. The term `odr` refers to uncertainty and variability in the observed default rate (`ODR`) at the reference date, while `lra` captures uncertainty and variability in the observed default rate over time.
Some functions, such as `odr_al`, indirectly include the effects of both sources. However, since they only implicitly capture one source of uncertainty, they are placed into a single group.

Practitioners should note that the implemented methods are not exhaustive, and some can be combined and further extended to develop new methods. Moreover, model-based and bootstrapping approaches are worth further investigation. As they are often difficult to generalize and integrate into function-ready procedures, they are not implemented in this package, but practitioners are encouraged to explore them for this purpose.

4 Case Study

4.1 Simulation Setup

This simulation study uses a dataset that can be imported by running:

```r
url.1 <- "https://raw.githubusercontent.com/andrija-djurovic/adsfcr/"
url.2 <- "main/risk_quantification/db_moc_c_egim_impact.csv"
url <- paste0(url.1, url.2)
db <- read.csv(file = url,
               header = TRUE)
```

The dataset contains 20 observations denoting number of years across two columns. Its structure can be viewed with:

```r
str(db)
```

```
## 'data.frame':    20 obs. of  2 variables:
##  $ year: int  1 2 3 4 5 6 7 8 9 10 ...
##  $ odr : num  0.0482 0.0295 0.0364 0.0361 0.0335 ...
```

where:

- `year` denotes the ordinal number of year for simulated `odr`;
- `odr` is the observed default rate.

As some of the implemented methods require the number of exposures for each reference date, in this case, we assume an equal number of exposures per year for simulation purposes. Practitioners should note that, in real-world applications, this is an observed input typically provided alongside the `ODR` and accompanying data.

The following code defines the number of exposures for each year.

```r
n <- rep(x = 1000, times = length(db$odr))
```

4.2 Statistical Method 1

This method assumes that the LRA default rate follows a normal distribution and accordingly calculates the upper bound of the selected confidence interval. Formally, this can be written as follows:

$$\text{LRA} + z_{1-\alpha}\frac{\sigma_{\text{ODR}}}{\sqrt{\text{n}}}$$

where:

- LRA is the long-run average calculated as the arithmetic average of reference dates' default rates;
- $z_{1-\alpha}$ is the $1-\alpha$ quantile of the standard normal distribution, where α is the significance level and $1-\alpha$ the selected confidence level;
- σ_{ODR} is the standard deviation of the observed reference dates' default rates;
- n is the number of observations (reference dates) used for the calculation of the LRA.

The following code estimates MoC based on this statistical method.

```
m1 <- lra_nf(odr = db$odr,
             cl = 0.95)
m1
```

```
##         lra         moc
## 1 0.0473259 0.004390624
```

4.3 Statistical Method 2

This method extends Method 1 by considering the effective sample size to account for the dependency of default rates over time. The following formula presents this adjustment:

$$\text{LRA} + z_{1-\alpha}\frac{\sigma_{\text{ODR}}}{\sqrt{\text{n}_e}}$$

where:

- LRA is the long-run average calculated as the arithmetic average of reference dates' default rates;
- $z_{1-\alpha}$ is the $1-\alpha$ quantile of the standard normal distribution, where α is the significance level and $1-\alpha$ the selected confidence level;
- σ_{ODR} is the standard deviation of the observed reference dates' default rates;
- n_e is the effective sample size obtained by dividing the actual length of the observed default rate series used for LRA calculation by $1+2\sum_{k=1}^{n}\text{AR}^k$, where AR is the first-order autocorrelation.

Before demonstrating the calculation of MoC based on this method, practitioners need to compute the so-called effective sample size factor. In this example, we assume a first-order autocorrelation process and define the factor under this assumption. However, practitioners should note that the same approach can be extended to different autocorrelation structures.

```
#calculation of the effective sample size factor
ols <- lm(formula = db$odr[-1] ~ db$odr[-length(db$odr)])
ar <- coef(ols)[2]
ess_factor <- 1 + 2*sum(ar^(1:length(db$odr)))
ess_factor
```

```
## [1] 1.778628
```

Given the calculation of the necessary input, the following code demonstrates the calculation of MoC based on this method.

```
m2 <- lra_ne(odr = db$odr,
             cl = 0.95,
             ess_factor = ess_factor)

m2
```

```
##         lra         moc
## 1 0.0473259 0.005855565
```

4.4 Statistical Method 3

This method is based on Jeffreys' confidence interval for a binomial proportion.
For the selected confidence level, Jeffreys' interval is given as:

$$\beta_{1-\alpha}(\text{shape1} = \text{ND} + 0.5, \text{shape2} = \text{N} - \text{ND} + 0.5)$$

where:

- $\beta_{1-\alpha}$ is the $1-\alpha$ quantile of the beta distribution with parameters shape1 and shape2;
- α is the significance level and $1-\alpha$ the selected confidence level;
- ND is the number of defaults at the reference date;
- N is the number of exposures at the reference date.

For the selected confidence level, this method calculates the upper bound of the binomial proportion and then recalculates the LRA. The difference between the observed and simulated LRA represents the MoC level.

The following code estimates MoC based on this statistical method.

```
m3 <- odr_jeffreys_ci(odr = db$odr,
                      n = n,
                      cl = 0.95)
m3
```

```
##         lra        moc
## 1 0.0473259 0.01193434
```

4.5 Statistical Method 4

Similar to Method 3, this approach is also based on a confidence interval for a binomial proportion, specifically the Clopper–Pearson interval.
For a selected confidence level, the Clopper–Pearson interval is given by:

$$\beta_{1-\alpha}(\text{shape1} = \text{ND} + 1, \text{shape2} = \text{N} - \text{ND})$$

where:

- $\beta_{1-\alpha}$ is the $1-\alpha$ quantile of the beta distribution with parameters shape1 and shape2;
- α is the significance level and $1-\alpha$ the selected confidence level;
- ND is the number of defaults at the reference date;
- N is the number of exposures at the reference date.

For the selected confidence level, this method calculates the upper bound of the binomial proportion and then recalculates the LRA. The difference between the observed and simulated LRA represents the MoC level.

The following code estimates MoC based on this statistical method.

```
m4 <- odr_clopper_pearson_ci(odr = db$odr,
                             n = n,
                             cl = 0.95)
m4
```

```
##         lra        moc
## 1 0.0473259 0.01248972
```

4.6 Statistical Method 5

This method is similar to Methods 3 and 4 but assumes a normal approximation of the binomial proportion distribution and accordingly derives the upper bound of the confidence interval.
Formally, this can be written as follows:

$$\text{ODR} + z_{1-\alpha}\sqrt{\frac{\text{ODR}\cdot(1-\text{ODR})}{N}}$$

where:

- ODR is the observed default rate at the reference date;
- $z_{1-\alpha}$ is the $1-\alpha$ quantile of the standard normal distribution, where α is the significance level and $1-\alpha$ the selected confidence level;
- N is the number of exposures at the reference date.

For the selected confidence level, this method calculates the upper bound of the binomial proportion and then recalculates the LRA. The difference between the observed and simulated LRA represents the MoC level.

The following code estimates MoC based on this statistical method.

```
m5 <- odr_na_ci(odr = db$odr,
                n = n,
                cl = 0.95)
m5
```

```
##         lra        moc
## 1 0.0473259 0.01095182
```

4.7 Statistical Method 6

This method is based on Monte Carlo simulation, assuming that the number of defaults follows a binomial distribution. Specifically, for a given ODR and number of exposures at each reference date, the method simulates the number of defaults for each reference date, recalculates the corresponding ODRs, and then recomputes the LRA based on the simulated values.
This process is repeated `sim` times (a parameter of the function below), and the percentile of the simulated LRA distribution is calculated. The percentile is controlled by the parameter `cl` in the function. Finally, MoC is defined as the difference between the simulated LRA percentile and the observed LRA.

The above process is implemented using the function `odr_mc` and can be run as follows.

```
set.seed(46)
m6 <- odr_mc(odr = db$odr,
             n = n,
             cl = 0.95,
             sim = 1e4)

m6
```

```
##         lra         moc
## 1 0.0473259 0.002524098
```

4.8 Statistical Method 7

This method is based on the normal distribution confidence interval of the LRA, given by the following estimator:

$$\text{LRA} = \frac{1}{T}\sum_{i=1}^{T} ODR_i$$

where:

- T is the number of reference dates;
- ODR_i is the observed default rate at reference date i.

The variance used for the calculation of the confidence interval for the above estimator is given by:

$$\frac{1}{T^2}\left(\text{LRA}\cdot(1-\text{LRA})\cdot\sum_{i}^{T}\frac{1}{n_i}\right)$$

where:

- T is the number of reference dates;
- n_i is the number of exposure at reference date i.

Finally, the MoC is estimated as the difference between the upper bound of the simulated confidence interval and the observed LRA.

The following code runs the above estimation method.

```
m7 <- odr_al(odr = db$odr,
             n = n,
             cl = 0.95)
m7
```

```
##         lra         moc
## 1 0.0473259 0.002469641
```

4.9 Statistical Method 8

This method implements independent bootstrapping of the ODRs. Specifically, each ODR has the same probability of being selected in the sample with replacement. After sampling the same number of ODRs as provided, the LRA is recalculated. This process is repeated `sim` times (a parameter of the function below), and a specified percentile of the simulated LRA distribution is then computed. The percentile is controlled by the parameter `cl` in the function. Finally, MoC is defined as the difference between the simulated LRA percentile and the observed LRA.
Practitioners should note that the bootstrapping approach can be extended to different designs that further account for dependency structures as well as potential heteroskedasticity in the observed default rates. However, due to the broad scope of these extensions, this package includes only the independent bootstrapping approach.

The above process is implemented using the function `lra_boot_ind` and can be run as follows.

```
set.seed(2105)
m8 <- lra_boot_ind(odr = db$odr,
                   cl = 0.95,
                   sim = 1e4)
m8
```

```
##         lra         moc
## 1 0.0473259 0.004323068
```

4.10 Statistical Method 9

The method is based on a Monte Carlo approach under the following assumptions:

1. One-year default rates follow the Vasicek distribution.
2. The systemic factor follows a first-order autoregressive process given by:

$$z_t = \phi \cdot z_{t-1} + \sqrt{1-\phi^2} \cdot \epsilon_t$$

   where z_t and ϵ_t are drawn from the standard normal distribution, while ϕ is estimated based on the ODRs.
3. Defaults within a given year follow a binomial distribution with a probability of success equal to the conditional PD given the realized systemic factor from the previous step.

Details on the Vasicek distribution and parameter estimation can be found here.

For demonstration purposes, we assume that the `pd` parameter is equal to the LRA, and the asset correlation parameter ρ is estimated using indirect moment matching method. Based on the estimated parameters of the Vasicek distribution, the required inputs for autocorrelation coefficient estimation are the observed systemic factors.

The code below calculates the necessary inputs for the demonstration of this statistical method.

```
#pd parameter
pd <- mean(db$odr)
pd
```

```
## [1] 0.0473259
```

```
#asset correlation (indirect moment matching method)
rho <- var(qnorm(p = db$odr)) / (1 + var(qnorm(p = db$odr)))
rho
```

```
## [1] 0.01475404
```

```
#systemic factor
z <- (qnorm(mean(db$odr)) - sqrt(1 - rho) * qnorm(db$odr)) / sqrt(rho)
z
```

```
##  [1] -0.17056653  1.67377784  0.90127486  0.93067496  1.20524961  1.43453010
##  [7] -1.06143702  0.47419419 -1.14219388 -0.99831109 -0.37161703 -1.23979490
## [13] -0.37305287  0.11563237 -1.58581739 -1.29130141  0.59377098 -0.11636761
## [19]  0.97411845 -0.05656202
```

```
#dependency between the realised systemic factors (ar(1) model)
ols <- lm(formula = z[-1] ~ z[-length(z)])
ar <- coef(ols)[2]
unname(ar)
```

```
## [1] 0.2945967
```

Given the estimated inputs, the following steps describe the simulation procedure:

1. For the estimated `pd`, ρ, and the autocorrelation structure of the systemic factor, simulate `T` (number of reference dates) values of conditional PDs from the Vasicek distribution.
2. Based on the conditional PDs and the number of exposures `n`, simulate the number of defaults and recalculate the PD for each year over the period `T`.
3. Calculate the simulated `pd` as the average of the PDs obtained in step 2.
4. Repeat steps 1 to 3 (a parameter of the function below) and store the results.
5. Calculate the specified percentile of the simulated `pd` distribution.
6. Compute MoC as the difference between the simulated `pd` percentile and the observed `pd`.

The following code implements the described simulation-based approach for MoC calculation.

```
set.seed(12)
m9 <- lra_odr_vasicek(pd = pd,
                      n = n,
                      rho = rho,
                      phi = ar,
                      cl = 0.95,
                      sim = 1e4)
m9
```

```
##         lra         moc
## 1 0.0473259 0.006474098
```

5 Conclusions

The following table presents a summary of all methods implemented in this package.

```
##   Method   lra      moc moc_relative
## 1     m1 4.73% 0.44 pp.        9.28%
## 2     m2 4.73% 0.59 pp.       12.37%
## 3     m3 4.73% 1.19 pp.       25.22%
## 4     m4 4.73% 1.25 pp.       26.39%
## 5     m5 4.73% 1.10 pp.       23.14%
## 6     m6 4.73% 0.25 pp.        5.33%
## 7     m7 4.73% 0.25 pp.        5.22%
## 8     m8 4.73% 0.43 pp.        9.13%
## 9     m9 4.73% 0.65 pp.       13.68%
```

As can be seen from the results, relative MoC levels range from slightly above 5% to above 26%, depending on the selected statistical method. Practitioners should note that some methods are highly impacted by the number of exposures per reference date; therefore, they are encouraged to simulate different scenarios and test these methods on real-world data.

Moreover, practitioners are encouraged to further investigate the impact of the underlying assumptions of each method, which is closely related to the sources of uncertainty incorporated. The presented methods and simulations should serve only as a starting point to help practitioners understand the potential impact of certain modeling choices. Fully addressing practical challenges requires further extensions beyond what is covered here.